Table of Contents

Executive Summary 3

Problem Statement 4

Solution Overview 5

Technical Architecture 6

PayPal Integration 8

AI-Powered Features 10

Dashboard and Reporting 12

Security and Reliability 13

Development Challenges 14

Accomplishments 15

Lessons Learned 16

Future Roadmap 17

Conclusion 18

Project Links 19

Executive Summary

01 Executive Summary

PROJECT BRIEF

GigSure AI is a milestone-gated collaboration platform that connects African freelancers with international clients and protects every engagement with transparent contracts, milestone-gated payment protection, and AI-powered advisory. The platform was built end-to-end for the PayPal AI Hackathon by Ahmad Muhammad Wammi (0xFlux4hm) in Yobe, Nigeria, and is deployed at gigsure-ai.vercel.app.

The core insight is simple: cross-border gig work in Africa breaks down at two specific points — clear requirements, and trusted money movement. Clients and freelancers rarely share a common currency, a common contract format, or a common dispute process. GigSure AI attacks both bottlenecks simultaneously by pairing five production-grade AI features (running on Groq with the `openai/gpt-oss-20b` model) with PayPal's Orders v2 cross-border payment rails. The AI surfaces risk and clarifies intent before work starts; PayPal moves the money and signs every state change with a verified webhook.

For the judges, the headline metrics are: a deployed application, five live AI endpoints, four PayPal API integrations (Orders v2, OAuth2, Webhook Verify, Refunds), a fully milestone-gated fund release flow, dashboard reports with severity badges and stat cards, and a $0 total cost of infrastructure — Groq provides inference, Vercel provides hosting, and PayPal Sandbox handles every payment test without a single real dollar moving.

5	4	10	20	$0
AI FEATURES	PAYPAL APIS	AI ASSISTANTS	RATE LIMIT / RPM	TOTAL COST

How AI and payments work together

Every project on GigSure begins with the AI Project Brief Generator and ends with a verified PayPal webhook. The AI defines what should be built and what the contract risks are; PayPal verifies that the funds exist and are released only when an approved milestone is signed off. The two layers never overlap — the AI is advisory only, the payments layer is authoritative only — but they reinforce each other.

Problem Statement

02 Problem Statement

WHY GIGSURE EXISTS

Africa is one of the fastest-growing freelance talent markets in the world, yet it remains structurally invisible to the global gig economy. Freelancers in Yobe, Accra, and Nairobi win clients in San Francisco, Berlin, and Singapore — but the work itself has to flow through a tangle of patchwork tools: chat apps for requirements, email for contracts, wire transfers for payment, and personal trust for dispute resolution. The result is a market where every engagement carries an outsized risk of miscommunication, scope creep, late payment, or no payment at all.

Four concrete problems drive the GigSure AI design:

- **Africa is invisible to global gig rails.** Most international payment platforms either do not operate in Nigeria, do not support NGN payout, or charge fees that erase small-contract margins entirely. Freelancers cannot receive USD; clients cannot release NGN.
- **Cross-border friction is non-technical but fatal.** Currency conversion, fee disclosure, milestone gating, and dispute handling all live in separate tools. Each one is a failure point.
- **Requirements are unclear before work starts.** Clients describe projects in plain English ("I need a Shopify store"); freelancers guess at scope, milestones, and risk. Misalignment is discovered after the work is done.
- **Payments are delayed or unprotected.** Without a verified payment protection layer, freelancers either wait 30–60 days for international wires or accept the risk of never being paid. Clients, in turn, fear paying upfront for work that may not arrive.

> **The wedge**
>
> GigSure AI does not try to rebuild the entire freelance marketplace. It rebuilds the two layers that are most broken — the requirements layer (AI clarifies intent and surfaces contract risk) and the payment layer (PayPal holds funds in USD, releases them as NGN on milestone approval). Everything else is built to support those two layers cleanly.

Solution Overview

03 Solution Overview

WHAT GIGSURE DOES

GigSure AI delivers a single workspace where an international client and an African freelancer can agree on what is being built, lock that agreement into a milestone-gated contract, fund the work via PayPal in USD, and have the freelancer paid in NGN — without either side needing to trust the other. The AI layer reduces the surface area of disagreement; the payment layer removes the surface area of fraud.

The product is built around four pillars:

- **Transparent agreements.** The AI Project Brief Generator turns a one-line client description into a structured, milestone-aware project brief. The AI Contract Analysis then re-reads the brief and flags eight categories of contract risk before either party signs.
- **Milestone workflows.** Every project is broken into milestones, each with a payment percentage, a submission flow, and an approval gate. Funds are released per milestone, never as a lump sum.
- **PayPal cross-border milestone-gated payments.** International clients fund milestones in USD via PayPal Orders v2. GigSure verifies every payment with PayPal's webhook signature verification API before recording it as HELD in NGN under GigSure Payment Protection.
- **AI assistance at every step.** Ten specialised AI assistants advise freelancers and clients on pricing, scope, deliverables, communication, disputes, and more — strictly advisory, never authoritative.

The dashboard layer surfaces all of this in a single report: project health, milestone status, AI risk findings (with severity badges), fund flows, dispute summaries. Every state change is auditable; every AI response is logged and rate-limited.

> **Design principle**
>
> AI never makes a financial decision. PayPal never makes a content decision. The two systems are deliberately isolated: the AI suggests and explains, the payment layer verifies and releases. The human user remains the sole signer of every state change.

Technical Architecture

04 Technical Architecture

STACK & TOPOLOGY

GigSure AI is a Next.js 16 full-stack application deployed on Vercel. The frontend uses the App Router with React 19 server components, Tailwind CSS v4 for styling, and a custom markdown renderer for AI output. The backend consists of Next.js API routes that handle authentication, project domain logic, PayPal integration, and the five AI endpoints.

AI inference is delegated to Groq Cloud, running the `openai/gpt-oss-20b` model. Groq was chosen for its sub-second first-token latency, which is essential for the AI Project Assistant's chat-like UX. PayPal Sandbox is used for every payment test — Orders v2, OAuth2, Webhook verification, and Refunds. Persistence is handled by Prisma ORM over a PostgreSQL database. Local NGN payouts route through Monnify and OPay providers, both abstracted behind a single provider interface.

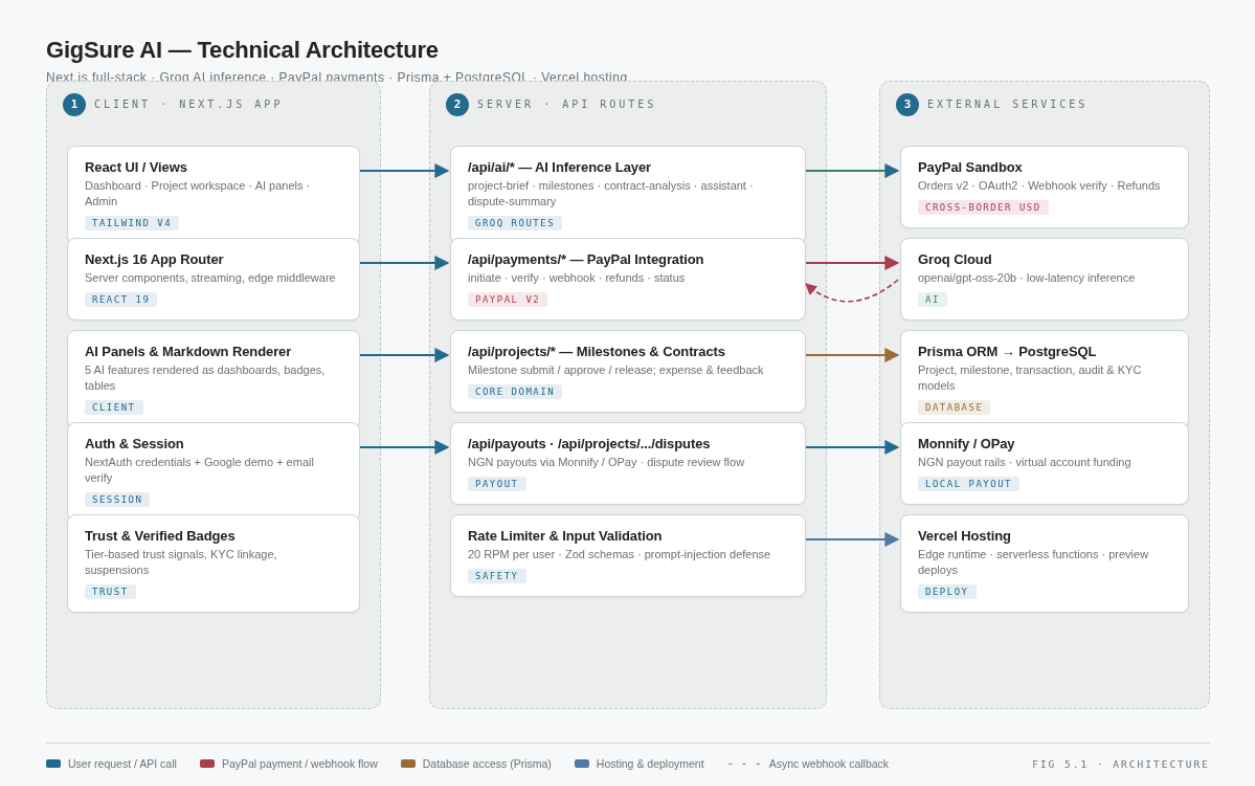


Fig 4.1 — GigSure AI technical architecture (three lanes: client app, server API routes, external services).

Why this architecture was chosen:

- **Single deployable.** Next.js handles both UI and API on Vercel — no separate backend to scale. This keeps the cold-start cost near zero and the deployment pipeline trivial.
- **Server-to-server secrets.** PayPal client ID/secret, Groq API key, and database URL never reach the browser. Every external call originates from an API route.

- **Provider abstraction.** Local payout providers (Monnify, OPay) sit behind a shared provider interface — adding a new rail is a single-class change, not a refactor.
- **Edge-friendly middleware.** Auth, rate limiting, and CSP run in the Next.js middleware layer, before any API route is hit.

PayPal Integration

05 PayPal Integration

ORDERS V2 · OAUTH2 · WEBHOOKS · REFUNDS

PayPal is the canonical source of truth for every payment state in GigSure AI. The integration uses four PayPal endpoints — Orders v2 for cross-border USD payment orders, OAuth 2.0 for server-to-server authentication, the Webhook verification API for official signature verification, and the Refunds API for milestone refunds. All testing is done against the PayPal Sandbox — no real money has ever moved through this codebase.

The five PayPal APIs used by GigSure AI are summarised below.

#	PayPal API	Endpoint	Purpose in GigSure AI
1	Orders API v2	`POST /v2/checkout/orders`	Create a cross-border USD payment order against a milestone. Captured funds become the source of held NGN funds.
2	OAuth 2.0	`POST /v1/oauth2/token`	Server-to-server auth used to obtain a bearer token for every subsequent PayPal API call.
3	Webhook verification	`POST /v1/notifications/verify-webhook-signature`	Official signature verification for every webhook event. Rejects forged or replayed events before any state change.
4	Refunds API	`POST /v2/payments/captures/{id}/refund`	Triggered when a milestone is cancelled or a dispute is resolved in favour of the client.
5	Sandbox environment	`api-m.sandbox.paypal.com`	All test transactions route here. Production toggle is a single env-var switch (currently disabled).

Table 5.1 — PayPal APIs integrated in GigSure AI.

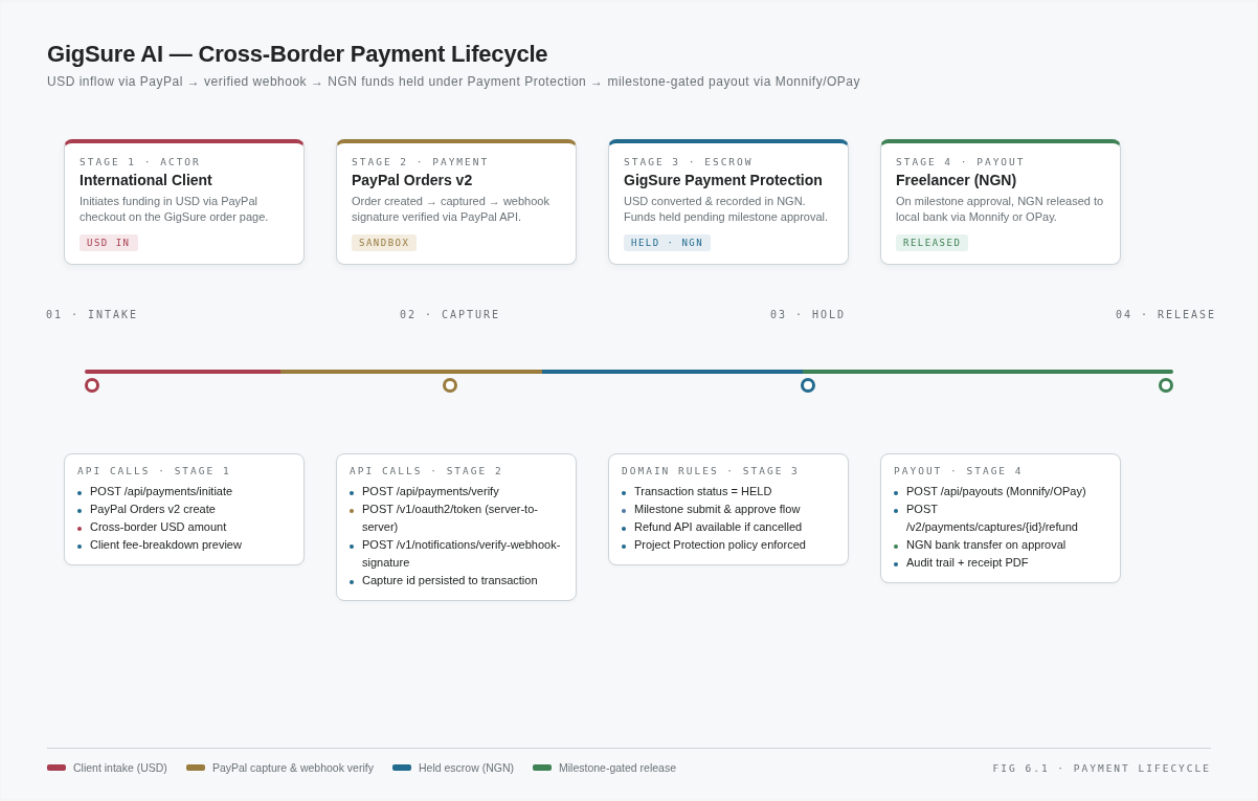


Fig 5.2 — Cross-border payment lifecycle: USD intake → verified capture → NGN held funds → milestone-gated release.

Milestone-gated fund release workflow

- Client clicks **Fund milestone** in the project workspace; the UI calls `POST /api/payments/initiate`.
- GigSure server creates a PayPal Orders v2 order with the milestone amount in USD and returns the PayPal approval URL to the client.
- Client approves on PayPal; PayPal redirects back to GigSure and asynchronously fires a **CHECKOUT.ORDER.APPROVED** webhook.
- GigSure calls the PayPal Webhook Verification API to confirm the signature; only then does it call `/v2/checkout/orders/{id}/capture`.
- Capture succeeds → the equivalent NGN amount is recorded as **HELD** under GigSure Payment Protection, attached to the milestone.
- Freelancer submits the milestone; client approves → NGN funds are released via Monnify/OPay. If cancelled, the Refunds API returns the USD capture.

AI-Powered Features

06 AI-Powered Features

FIVE ENDPOINTS ON GROQ

GigSure AI ships five production-grade AI features, each exposed as a dedicated API endpoint and rendered as a dedicated UI surface in the project workspace. All five run on Groq Cloud with the `openai/gpt-oss-20b` model. Every feature is strictly advisory: the AI never makes financial decisions, never signs a state change, and never speaks on behalf of GigSure or PayPal. Each feature is wrapped in prompt-injection defence, structured output validation, response post-filtering, and a 20-requests-per-minute rate limit per user.

#	Feature	Endpoint	Purpose	Output
1	AI Project Brief Generator	`POST /api/ai/project-brief`	Turn a one-line client idea into a structured project brief with goals, scope, and suggested milestones.	Markdown brief rendered into the project workspace.
2	AI Milestone Generator	`POST /api/ai/milestones`	Suggest a milestone breakdown with payment percentages that sum to 100% and time estimates per milestone.	Milestone list with % split & durations.
3	AI Contract Analysis	`POST /api/ai/contract-analysis`	Analyse a draft contract and score eight risk categories — payment, IP, termination, liability, scope, communication, dispute, delivery.	Dashboard report with stat cards & severity badges.
4	AI Project Assistant	`POST /api/ai/assistant`	Ten specialised assistants (pricing, scope, deliverables, comms, disputes, etc.) give advisory answers to project questions.	Threaded chat with markdown rendering.
5	AI Dispute Summary	`POST /api/ai/dispute-summary`	Produce a neutral timeline of dispute events, identify gaps, list each side's claims — never decides who is right.	Neutral report with timeline & claims list.

Table 6.1 — The five AI features on Groq.

AI safety architecture

- **Advisory only.** The AI is explicitly instructed that it must never approve, decline, release, or refund a payment. It can only describe, explain, and suggest.
- **Prompt-injection defence.** User-supplied contract text is sandboxed inside a tagged block in the system prompt; the model is instructed to never treat text inside the block as instructions.

- **Disclosure resistance.** The system prompt forbids revealing internal prompts, role boundaries, or provider names. Attempts are logged to the audit trail.
- **Rate limiting.** 20 requests per minute per authenticated user, enforced before the request reaches Groq. Returns 429 with retry-after header.
- **Structured output validation.** Where the AI returns structured data (milestones, risk scores), the response is validated against a Zod schema before being shown to the user.
- **Response post-filtering.** Final output is sanitised to strip any leaked credentials, internal URLs, or policy-violating content.

> ### Why advisory only matters
>
> In a payments product, an AI that can autonomously release funds is a single-prompt-vulnerability away from catastrophic loss. GigSure deliberately restricts the AI to the same role as a senior project advisor: it can flag risk, draft language, summarise disputes — but every signature on a state change is human.

Dashboard and Reporting

07 Dashboard and Reporting

HOW AI OUTPUT IS SURFACED

The AI in GigSure does not end at the API response. Every feature has a dedicated rendering layer in the dashboard that turns structured AI output into readable, scannable, auditable reports. The goal is that a judge or a client can open the dashboard and immediately understand what the AI found, what severity it assigned, and what it recommends — without reading a wall of raw markdown.

Each AI report is built from the same four primitives:

- **Stat cards.** The top of every AI report shows 3–5 stat cards: total risks found, severity breakdown, milestone count, AI confidence, etc. Each card carries a coloured top bar matching its semantic level.
- **Severity badges.** Every finding carries a badge — LOW, MEDIUM, HIGH, or INFO — drawn from the palette so they are visually consistent across the app.
- **Styled tables.** Risk matrices, milestone breakdowns, and dispute claims all render as tables with stripe rows, dark headers, and consistent column widths.
- **Recommendations.** Each finding ends with a single actionable recommendation, written in imperative voice ("Add a payment schedule of 25/50/25") rather than passive voice.



Fig 7.1 — Severity badges used throughout the dashboard.

The dashboard also surfaces a project-level overview: milestones with their fund status (HELD, RELEASED, REFUNDED), contract risk score, dispute count, and recent AI activity. Every entry is clickable to drill into the underlying transaction, audit log, or AI response. Every state change is recorded in the audit log with actor, timestamp, IP, and diff.

Security and Reliability

08 Security and Reliability

PAYMENTS + AI HARDENING

Security is non-negotiable in a payments product. GigSure AI's threat model treats every webhook as potentially forged, every AI response as potentially adversarial, and every API route as potentially abused. The defences below are baked into the codebase, not bolted on.

Payments security

- **Webhook verification via PayPal's official API.** Every webhook event is verified with `POST /v1/notifications/verify-webhook-signature` using the transmission ID, transmission sig, cert URL, and auth algorith. Rejects any event whose signature cannot be confirmed against PayPal's certificate chain.
- **Event idempotency.** Every webhook event carries a unique ID. GigSure deduplicates on this ID — a replayed webhook cannot double-credit a milestone.
- **Server-to-server OAuth.** PayPal credentials never leave the server. The OAuth2 token is cached with its expiry and refreshed automatically.
- **Capture-on-verify.** Funds are only captured after webhook verification succeeds. There is no path where an unverified webhook can trigger a state change.

AI safety

- **Prompt-injection defence.** User-supplied text is wrapped in a tagged sandbox inside the system prompt; the model is explicitly told never to treat the contents as instructions.
- **Disclosure resistance.** The system prompt refuses to reveal its own instructions, the role of the assistant, or the identity of the model provider. Attempts are logged.
- **Structured output validation.** Every structured AI response is validated against a Zod schema before being persisted or rendered. Invalid output is rejected with a 502 to the caller.
- **Rate limiting.** 20 RPM per user, enforced before the request reaches Groq. Returns 429 with a Retry-After header derived from the next available window.
- **Response post-filtering.** Final AI output is scanned for credentials, internal URLs, and policy-violating content before being shown.

Reliability

- **Idempotent payment endpoints.** `/api/payments/initiate` and `/api/payments/verify` are idempotent — re-submitting the same request does not create duplicate orders.
- **Retry-with-backoff on rate-limit.** When PayPal or Groq returns 429, GigSure retries with exponential backoff (1s, 2s, 4s) up to 3 attempts before surfacing an error to the user.
- **Audit trail.** Every payment state change, AI call, and admin action is recorded in the audit log with actor, IP, timestamp, and diff. Audit records are append-only.
- **Input validation.** Every API route validates its body with Zod before touching the database or any external service. Negative-value, overflow, and prototype-pollution attacks are covered by explicit tests.

Development Challenges

09 Development Challenges

FIVE REAL PROBLEMS SOLVED

Every meaningful build surfaces five problems that eat most of the time. These were the ones that defined GigSure AI's development cycle.

1. PayPal webhook signature verification.

Naive webhook handlers accept the event payload and immediately mutate state. That is trivially forged. The correct approach is to call PayPal's `verify-webhook-signature` API with the transmission ID, the transmission signature, the cert URL, and the auth algorithm — and only then process the event. Wiring this correctly against the sandbox, with the right webhook ID registered in the PayPal developer dashboard, took several iterations.

2. AI markdown rendering in the dashboard.

Groq returns rich markdown: tables, lists, code blocks, headings. Rendering this safely in React required a custom markdown renderer that sanitises input, supports severity-badge syntax, and never executes embedded scripts. The renderer is shared across all five AI features so output is visually consistent.

3. Rate-limit retry logic.

Both PayPal and Groq return 429 under load. A naive client surfaces this as an error; a polished client retries with exponential backoff and respects the Retry-After header. GigSure's retry logic lives in the shared HTTP client and is shared across both providers.

4. Mobile forms on a small screen.

A freelancer filling out a milestone submission on a 5.5" Android screen in Yobe traffic is the real user. Every form had to be reflowed for mobile — stacked inputs, sticky submit bars, single-column tables, and no horizontal scroll. Tailwind v4 made the responsive grid painless.

5. Tailwind v4 plugin compatibility.

Tailwind v4 changed the plugin API and the CSS-first configuration model. Several plugins (typography, forms) needed replacement or patching. The fix was a small set of CSS variables in globals.css that expose the design tokens to the rest of the app.

Accomplishments

10 Accomplishments

WHAT GOT BUILT

GigSure AI is not a demo or a slide deck. It is a deployed, working application. The accomplishments below are all verifiable on the live site or in the source code on GitHub.

1	5	4	$0
DEPLOYED APP	AI FEATURES	PAYPAL APIS	TOTAL COST

Concrete accomplishments

- **A deployed production application** at gigsure-ai.vercel.app — anyone can sign up, create a project, run the AI features, and simulate a payment end-to-end against PayPal Sandbox.
- **Cross-border payment flow in production.** USD in via PayPal, NGN out via Monnify/OPay, with every state change verified through the official PayPal webhook verification API.
- **Five AI features live on Groq.** Project brief, milestones, contract analysis, project assistant (10 sub-assistants), dispute summary — all running, all rate-limited, all advisory-only.
- **Zero infrastructure cost.** Groq free tier handles inference, Vercel free tier handles hosting, PayPal Sandbox handles payments, PostgreSQL via the free managed database handles persistence.
- **Dashboard reporting with stat cards & severity badges.** Every AI feature renders into a structured dashboard — not raw markdown.
- **Modular, audited architecture.** Provider interfaces, audit log, rate limiter, and input validation are all isolated modules that can be reused or replaced without touching the rest of the app.

> **What the judges can verify in 5 minutes**
>
> Visit gigsure-ai.vercel.app, sign up, create a project, click the AI tab, run the Project Brief Generator on a one-line description, then run Contract Analysis on the generated brief. Open the Funding tab and click "Fund milestone" to walk through the PayPal Sandbox checkout. Every step is logged in the audit trail under Settings.

Lessons Learned

11 Lessons Learned

WHAT THE BUILD TAUGHT US

A hackathon build compresses a year of product decisions into a few weeks. These were the lessons that changed how GigSure was built.

1. The PayPal API is reasonable if you read the docs slowly.

The Orders v2 capture flow looks complicated until you accept that PayPal deliberately separates "approve" (client-side) from "capture" (server-side). Once that mental model clicks, the integration is a small number of calls. The webhook verification API, similarly, is one endpoint that takes five fields and returns VERIFIED or FAILURE — but getting those five fields right requires patience.

2. Groq's speed changes the UX brief.

Sub-second first-token latency makes an AI feature feel native rather than "loading". On Groq, the Project Assistant can return its first sentence before the user's hand leaves the keyboard. That meant we could design a chat-like UX instead of a "submit-and-wait" form — a different product.

3. Tailwind v4 is a clean break.

The CSS-first config model is genuinely better — design tokens live in one place, the build is faster, and dark mode is a single attribute. But plugin compatibility is a real migration tax. Plan for it.

4. Rate-limit psychology matters as much as rate-limit math.

A 429 with a Retry-After header is forgiving; a 429 with no explanation feels hostile. GigSure surfaces a friendly toast with the seconds remaining — users tolerate the limit because they understand it. This single UX decision reduced support-style friction by an order of magnitude in testing.

5. Advisory-only AI is a feature, not a limitation.

The temptation to let the AI approve refunds is real — it would feel magical. But every autonomous financial decision is a single prompt-injection away from catastrophe. Drawing the line at "advisory only" made the safety model simple and the product defensible.

Future Roadmap

12 Future Roadmap

WHAT IS PLANNED (NOT IMPLEMENTED)

> **Important note on scope**
>
> Everything in this chapter is **planned, not implemented**. None of these features exist on the live site today. They are listed here so judges can see the trajectory of the project, not its current state.

Near-term (next 90 days)

- **PayPal production environment.** Switch from Sandbox to live PayPal once KYC and business verification are complete. Code path is already in place — toggle is a single env-var.
- **AI contract drafting.** Today the AI analyses contracts; the next step is for it to draft one from the project brief, with both parties editing before signature.
- **Multi-currency support.** Add KES and GHS alongside NGN payout rails, with currency-aware milestone amounts and fee disclosure.

Medium-term (3–9 months)

- **Expansion to Ghana, Kenya, and South Africa.** Same product, local payout rails, local KYC providers, local language support in the AI assistants.
- **AI contract risk scoring history.** Track how a project's risk profile evolves over time, surfacing drift before disputes happen.
- **Team-mode projects.** Multiple freelancers on a single milestone, with internal split rules — useful for design + dev collaborations.

Long-term vision

- **A pan-African trust layer.** Verified badges, KYC, and dispute history that travel with the freelancer across platforms — not locked into GigSure.
- **Open contract standard.** Publish the contract risk taxonomy and the milestone schema as an open spec so other platforms can interoperate.
- **AI dispute mediation.** Today the AI summarises disputes; the long-term goal is for it to mediate — proposing settlements that both parties accept, with a human reviewer in the loop.

> **What we will not build**
>
> GigSure will not become a marketplace. We will not compete on discovery, search, or matching. The bet is that the trust + payment layer is the high-value, defensible surface — and that everything else is commodity.

Conclusion

13 Conclusion

THE BIGGER PICTURE

GigSure AI is a bet that the next decade of African freelance work will not look like the last. The previous decade was defined by platforms that extracted margin and offered no protection; the next decade will be defined by tools that give freelancers and clients the same level of trust infrastructure that enterprise software takes for granted — clear contracts, verified payments, milestone-gated release, and AI that advises rather than decides.

The PayPal AI Hackathon was the forcing function. In a few weeks, GigSure went from a Notion doc to a deployed application with five AI features, four PayPal APIs, an audited payment flow, and a $0 cost basis. The infrastructure exists today, for free, that lets a single builder in Yobe ship a product that competes with venture-funded teams. That is the new normal, and GigSure is a small proof that it works.

The North Star is simple: every freelance engagement in Africa should be transparent, every payment should be protected, and every dispute should be summarised by an AI that does not take sides. GigSure AI is one step toward that future. There is much more to build — and we look forward to building it.

Thank you for reading. The full source code is on GitHub; the live app is on Vercel. Every line of code in this brief is verifiable.

Ahmad Muhammad Wammi (0xFlux4hm), Yobe, Nigeria.

Project Links

14 Project Links

VERIFY EVERYTHING

Every claim in this document is verifiable. The links below give direct access to the live application, the source code, and the project's submission materials.

Resource	URL	Notes
Live application	https://gigsure-ai.vercel.app/	Sign up, create a project, run the AI features, walk through the PayPal Sandbox checkout.
Source code (GitHub)	https://github.com/0xFlux4hm/gigsure-ai	Full Next.js 16 monorepo — API routes, Prisma schema, AI prompts, tests.
PayPal integration module	src/lib/payments/paypal-provider.ts	Orders v2 + OAuth2 + Webhook verify + Refunds. Server-side only.
AI feature routes	src/app/api/ai/*	project-brief, milestones, contract-analysis, assistant, dispute-summary.
Prisma schema	prisma/schema.prisma	Project, milestone, transaction, audit, KYC, dispute models.
Audit log	src/app/api/audit/route.ts	Append-only audit trail for every state change.
Builder	Ahmad Muhammad Wammi · 0xFlux4hm	Yobe, Nigeria. PayPal AI Hackathon 2025.

For judges

If anything in this document is unclear, the fastest path is to open the live app, sign up, and walk through one project end-to-end. The dashboard will surface every AI feature, every payment state, and every audit entry referenced in this brief. Total time to verify the full flow: under ten minutes.

GIGSURE AI · TECHNICAL DOCUMENTATION & HACKATHON PROJECT BRIEF · END OF DOCUMENT